

never seen. The machinery below is how.

If you read nothing else in Part 2, read this: attention = each token mixing in a weighted blend of the other tokens. Q, K, V, softmax and the rest are the plumbing that decides the weights.

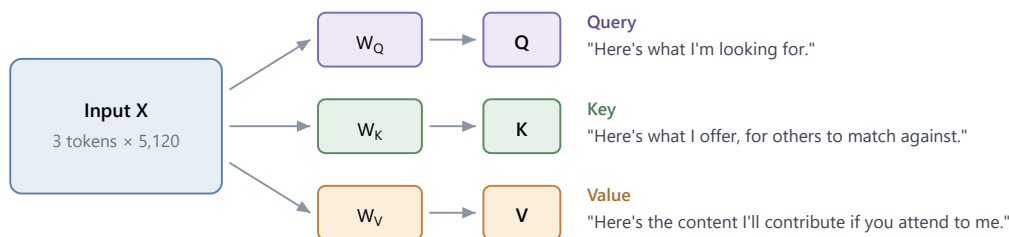
Attention, Step by Step

Vectors go in, the same number of vectors come out, each the same size — but every one has now absorbed some of its neighbours. Nothing about the shape changes. Only the contents.

To work out the blending percentages, the model gives each token three different jobs to do at once. It makes three new vectors out of each token's embedding, by multiplying it by three learned weight matrices. Same input, three different outputs, because the three matrices learned three different things:

- **Query** — what this token is looking for.
- **Key** — what this token can offer to others.
- **Value** — what this token actually hands over if someone picks it.

Queries and keys exist only to work out the percentages. Values are the stuff that actually gets blended.



Think of a filing cabinet: the query is your sticky note, the keys are folder labels, the values are the contents.

Three projections of the same input. Q asks, K advertises, V delivers.

Step 1 — Score every pair

Take one token's query and compare it against every token's key. "Comparing" here means a dot product: multiply the two lists position by position and add up the results. A big number means the query and key point the same way — this token is relevant to me. A small number means it isn't.

Do that for every query against every key and you get a grid of scores, one per pair. In matrix form that's QK^T , but it's just "everyone scores everyone."

Step 2 — Shrink the scores

Divide every score by $\sqrt{d_k}$ — the square root of the head width, so $\sqrt{128} \approx 11.3$ here. Why: adding up 128 products gives big numbers, and the next step (softmax) reacts badly to big numbers — it slams everything to 0 or 1 and the model stops learning. Dividing keeps the scores in a range softmax can work with. Pure housekeeping, but skip it and training fails.